HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

ĐỀ TÀI

Xây dựng hệ CSDL lưu trữ và tìm kiếm văn bản
(Nội dung 1 và 2)

Môn học: Hệ cơ sở dữ liệu đa phương tiện

Mã môn học: INT1418

Lớp: D23 - Lớp 01

Nhóm: 27

Đỗ Hoàng Minh MSV:B22DCPT166

Nông Mạnh Dũng MSV:B23DCCN206

Giảng viên: Nguyễn Đình Hóa

HÀ NỘI, 10/2026

Mục lục

1 Xây dựng và mô tả bộ dữ liệu **1**

1.1 Tổng quan bộ dữ liệu 1

1.2 Quy trình xây dựng 1

1.3 Chủ đề 2

1.4 Đặc điểm nội dung 3

2 Nguyên lý xây dựng đặc trưng và cách tìm kiếm văn bản **5**

2.1 Tổng quan 5

2.2 Tiền xử lý văn bản 5

2.3 Cụm từ cố định 8

2.4 Xây dựng từ điển 11

2.5 Trọng số TF-IDF 12

2.6 Biểu diễn văn bản trong không gian vector 15

2.7 Độ tương đồng cosine 16

2.8 Quy trình tìm kiếm văn bản 17

Danh sách hình vẽ

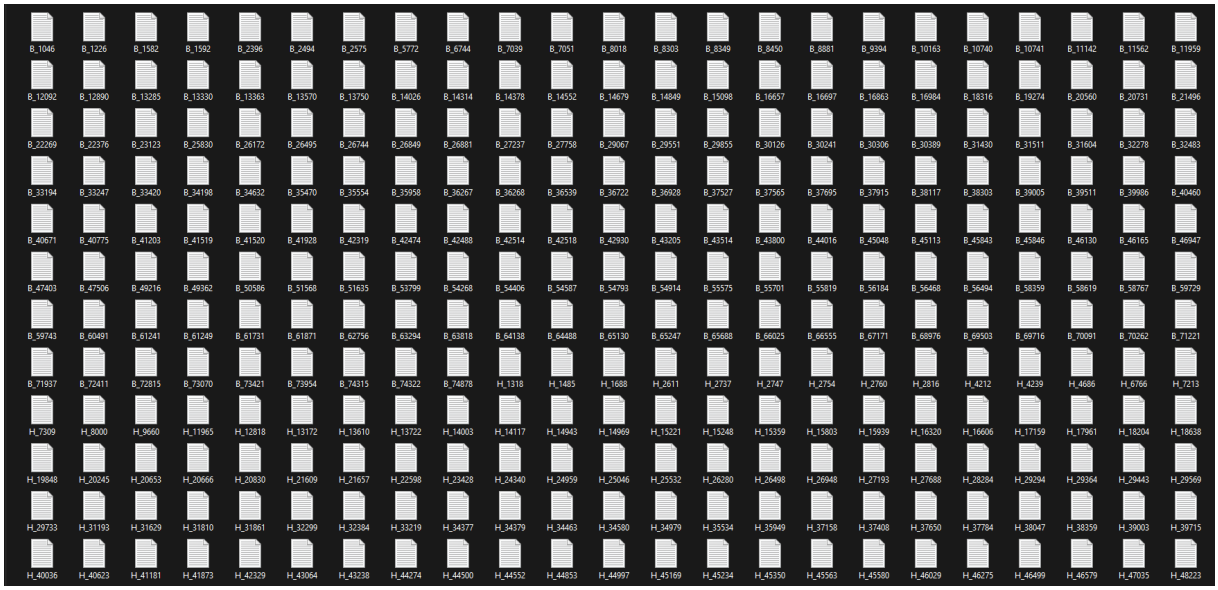
1.1 Một phần danh sách file trong bộ dữ liệu 1
1.2 File B_48001 4
1.3 File R_43300 4

2.1 Các hàm tiền xử lý 7
2.2 File B_48001 sau tiền xử lý 8
2.3 File R_43300 sau tiền xử lý 8
2.4 Hàm tìm cụm từ cố định bằng NPMI 11

Chương 1: Xây dựng và mô tả bộ dữ liệu

1.1. Tổng quan bộ dữ liệu

- **Nguồn:** Sách từ Project Gutenberg, lấy từ bộ dữ liệu trên Kaggle gồm khoảng 75.000 cuốn, kèm bảng thông tin của từng cuốn (tiêu đề, tác giả, ngôn ngữ, mã phân loại):
  https://www.kaggle.com/datasets/lokeshparab/gutenberg-books-and-metadata-2025
- **Quy mô:** 1.080 file .txt (bảng mã UTF-8), mỗi file dài 12.000 – 15.000 từ.
- **Tên file:** chữ cái của chủ đề + mã sách trên Gutenberg, ví dụ *B_10163.txt*.



Hình 1.1: Một phần danh sách file trong bộ dữ liệu

1.2. Quy trình xây dựng

Mỗi cuốn sách trong bảng thông tin có một mã phân loại theo hệ thống của Thư viện Quốc hội Mỹ (Library of Congress Classification, LoCC); chữ cái đầu của mã cho biết lĩnh vực lớn, ví dụ B là Triết học – Tâm lý – Tôn giáo. Văn bản được lấy qua 4 bước:

1. **Chọn theo bảng thông tin:** Sách tiếng Anh, thuộc một trong 6 lĩnh vực B, H, N, R, S, M; có đúng một mã LoCC, vì sách nhiều mã thuộc nhiều chủ đề cùng lúc; không trùng tiêu đề, tức các tập của một bộ sách chỉ giữ một tập.

2. **Làm sạch:** Chỉ bỏ những gì Project Gutenberg thêm vào (lời giới thiệu, giấy phép, ghi công, mốc trang, ký hiệu định dạng), giữ nguyên mọi thứ tác giả viết, kể cả chú thích.

3. **Cắt đoạn:** Lấy tối đa 15.000 từ liên tục ở giữa sách, bỏ số từ bằng nhau ở đầu (bìa, mục lục, lời tựa) và ở cuối (phụ lục, bảng tra cứu). Ví dụ, sách 50.000 từ thì bỏ 17.500 từ đầu và 17.500 từ cuối. Điểm cắt được dời tới ranh giới đoạn văn gần nhất để không cắt ngang câu.

4. **Kiểm tra:** Loại sách vi phạm một trong 4 luật ở bảng dưới. Ngoài ra, mỗi tác giả có tối đa 3 cuốn trong một chủ đề để đa dạng nội dung và vốn từ.

Luật	**Căn cứ**
1. Dưới 12.000 từ sau làm sạch	Yêu cầu của đề bài
2. Đoạn trích còn dấu hiệu của Gutenberg	Phần của Gutenberg giống hệt nhau ở mọi sách, làm các file giống nhau giả tạo
3. Không phải văn xuôi tiếng Anh: *the, of, and, to, in* chiếm dưới 10% số từ	Văn xuôi tiếng Anh có 15 – 25% các từ này; văn bản ngôn ngữ khác, bảng hay danh sách gần như không có
4. Trùng nội dung	Hai bản của cùng một cuốn sách

Bảng 1.1: Bốn luật loại sách

1.3. Chủ đề

Bộ dữ liệu gồm 6 chủ đề, mỗi chủ đề là trọn một lĩnh vực LoCC và gồm 180 file.

Trong mỗi chủ đề, sách thuộc nhiều lớp con, nên mỗi chủ đề bao quát nhiều khía cạnh:

Mã	Chủ đề	Tác giả	Số từ TB	Từ / câu	Từ khác nhau TB
B	Triết học – Tâm lý – Tôn giáo	163	14.829	25,9	2.670
H	Khoa học xã hội	162	14.869	25,4	2.739
N	Mỹ thuật	154	14.823	23,7	2.902
R	Y học	162	14.883	24,6	2.639
S	Nông nghiệp	134	14.853	23,5	2.497
M	Âm nhạc	144	14.850	23,3	2.904
Toàn bộ		**894**	**14.851**	**24,4**	**2.725**

Bảng 1.2: Thống kê theo chủ đề

Mã	Ba lớp con có nhiều file nhất (số file)
B	BX – các giáo phái Kitô giáo (34), BF – tâm lý học (26), BV – thần học thực hành (23)
H	HV – tệ nạn xã hội, phúc lợi (45), HQ – gia đình, hôn nhân, phụ nữ (37), HD – kinh tế công nghiệp (27)
N	ND – hội họa (52), N – mỹ thuật chung (48), NK – nghệ thuật trang trí (40)
R	RC – nội khoa (54), RA – y tế công cộng (36), RM – trị liệu, dược học (28)
S	SB – trồng trọt (74), SF – chăn nuôi (40), S – nông nghiệp chung (29)
M	ML – văn học về âm nhạc (131), MT – lý thuyết và dạy nhạc (44), M – âm nhạc chung (5)

Bảng 1.3: Lớp con LoCC trong từng chủ đề

1.4. Đặc điểm nội dung

- **Thể loại:** Gần như toàn bộ là văn xuôi trình bày kiến thức: sách chuyên khảo, giáo trình, tiểu luận, bài giảng, hồi ký. Mỗi file là một đoạn liên tục ở giữa một cuốn sách, nên thường nói về một đề tài hẹp trong chủ đề, ví dụ một bệnh cụ thể trong Y học hay một nhạc sĩ trong Âm nhạc.

- **Ngôn ngữ:** Sách đã hết bản quyền nên Tiếng Anh thường là văn phong cũ: câu dài, nhiều cách viết cổ như *thee, thou, hath* trong sách tôn giáo.

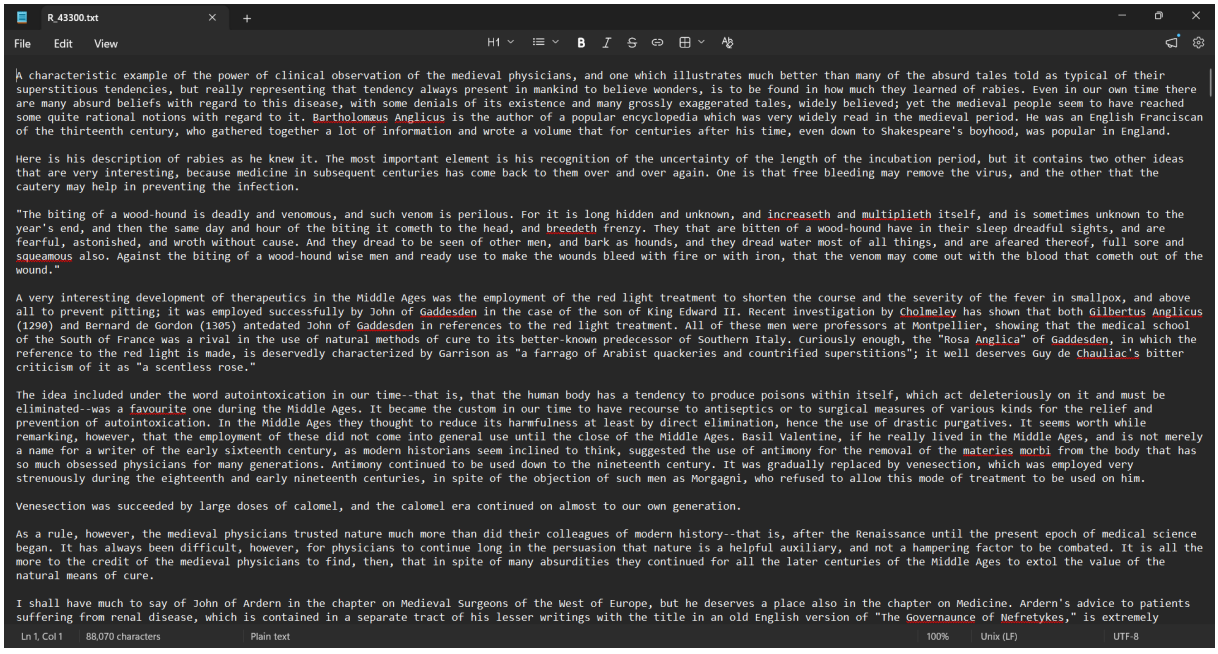
- **Từ vựng:** Mỗi chủ đề có nhóm từ riêng rõ rệt, cụ thể hơn ở bảng 2.6.

- **Đa dạng tác giả:** 1.080 file đến từ 894 tác giả, mỗi tác giả có tối đa 3 cuốn trong một chủ đề, nên mỗi chủ đề gồm nhiều văn phong và góc nhìn khác nhau.
- **Chủ đề có phần chồng lấn:** Một số cặp chủ đề có nội dung gần nhau, ví dụ Tôn giáo với Khoa học xã hội (đạo đức, xã hội), Khoa học xã hội với Y học (y tế công cộng).

Sau đây là một số file trong bộ dữ liệu:



Hình 1.2: File B_48001



Hình 1.3: File R_43300

Chương 2: Nguyên lý xây dựng đặc trưng và cách tìm kiếm văn bản

2.1. Tổng quan

Máy tính không đọc hiểu được văn bản để biết hai văn bản có giống nhau về nghĩa hay không. Vì vậy, mỗi văn bản được biểu diễn thành một vector số, gọi là vector đặc trưng. Khi mọi văn bản đã là vector trong cùng một không gian, câu hỏi "hai văn bản giống nhau đến mức nào" trở thành câu hỏi "hai vector gần nhau đến mức nào" và máy tính tính được điều đó.

Hệ thống sử dụng:

- Tiền xử lý và chỉ số NPMI để biến văn bản thành danh sách từ khóa, gồm cả các cụm từ cố định.
- TF-IDF và chuẩn hóa L_2 để xây dựng vector đặc trưng cho từng văn bản.
- Độ tương đồng cosine để so sánh văn bản truy vấn với các văn bản trong bộ dữ liệu.

2.2. Tiền xử lý văn bản

Văn bản sau khi trích vẫn chưa thể dùng trực tiếp để so sánh vì:

- **Cùng một từ có nhiều dạng viết:** *Disease*, *disease* và *diseases* nói cùng một khái niệm, nhưng nếu giữ nguyên thì máy tính coi đó là ba từ khác nhau.
- **Phần lớn các từ không mang nội dung:** Các từ như *the, of, and, is* chiếm khoảng một nửa số từ của mọi văn bản, bất kể chủ đề. Nếu giữ lại, chúng làm mọi văn bản trông giống nhau và lấn át các từ thật sự nói lên nội dung.
- **Có nhiều thành phần không phải từ:** Chữ số, dấu câu, số thứ tự chương mục cũng không giúp phân biệt chủ đề.

Tiền xử lý đưa mọi văn bản về một danh sách từ khóa thống nhất, chỉ gồm các từ mang nội dung, viết theo một dạng duy nhất. Đầu vào của bước này là các file văn bản đã trích, đầu ra là danh sách từ khóa của từng văn bản, các bước cụ thể như sau:

1. **Tách câu:** Tách tại dấu chấm, chấm hỏi, chấm than, chấm phẩy, hai chấm. Ranh giới câu được giữ lại để bước 6 không ghép hai từ ở hai câu khác nhau.

2. **Tách từ, bỏ chữ số và dấu câu:** Chỉ giữ các chuỗi chữ cái. Chữ có dấu được đưa về chữ không dấu (*Händel* thành *Handel*).

3. **Đưa từ về dạng gốc:** *studies, studied, studying* thành *study*; *was, were* thành *be*. Bộ gán từ loại của thư viện NLTK cho biết mỗi từ trong câu là danh từ, động từ, tính từ hay trạng từ, rồi từ điển WordNet trả về dạng gốc theo đúng từ loại đó. Ví dụ *leaves* trong "the leaves fall" là danh từ nên thành *leaf*, trong "he leaves early" là động từ nên thành *leave*.

4. **Chuyển về chữ thường:** *Alice*, *ALICE* và *alice* được tính là cùng một từ. Bước này làm sau bước 3, vì chữ hoa giúp bộ gán từ loại nhận ra tên riêng.

5. **Bỏ từ dừng:** Từ dừng là các từ không mang nội dung, gồm 224 từ:

   - 198 từ dừng tiếng Anh của NLTK: *the, of, and, is, just*...
   - 18 từ chức năng của tiếng Anh cổ: *thee, thou, hath, unto, shall*...
   - 8 từ chỉ cấu trúc sách: *chapter, chap, footnote, preface, page, vol, ibid, idem*. Các từ cấu trúc có nghĩa thứ hai trong các chủ đề thì được giữ lại, ví dụ *appendix* (ruột thừa), *volume* (âm lượng, thể tích), *illustration* (tranh minh họa).

   Ngoài ra bỏ các từ chỉ có 1 – 2 chữ cái (phần lớn là chữ viết tắt như *st, mr*) và số La Mã như *iii, vii, xiv* (đánh số chương mục, giống chữ số).

6. **Ghép cụm từ cố định:** Hai từ liền nhau nằm trong danh sách cụm từ cố định được ghép thành một từ khóa, ví dụ *nervous system* thành *nervous_system*. Cách lập danh sách này được trình bày ở mục tiếp theo.

Ví dụ: Câu "*CHAPTER VII. The diseases of the nervous system were studied in 1890.*" qua từng bước được trình bày ở Bảng 2.1.

Bước	**Kết quả**
Văn bản gốc	*CHAPTER VII. The diseases of the nervous system were studied in 1890.*
1. Tách câu	Câu 1: *CHAPTER VII.* Câu 2: *The diseases of the nervous system were studied in 1890.*
2. Tách từ, bỏ số, dấu câu	*CHAPTER VII The diseases of the nervous system were studied in*
3 – 4. Dạng gốc, chữ thường	*chapter vii the disease of the nervous system be study in*
5. Bỏ từ dừng, số La Mã	*disease nervous system study*
6. Ghép cụm từ	*disease nervous_system study*

Bảng 2.1: Ví dụ tiền xử lý một câu

```python
def chuan_bi(vb):
    global _tagger
    if _tagger is None:
        _tagger = PerceptronTagger()
    vb = bo_dau(chuan_hoa_ky_tu(vb))
    ds = []
    for cau in CAU.split(vb):                          # 1. tách câu
        tu = TU.findall(cau)                           # 2. tách từ, bỏ số, dấu câu
        if tu:
            ds.append([lemma(w.lower(), tu_loai(t))    # 3-4. dạng gốc, chữ thường
                       for w, t in _tagger.tag(tu)])
    return ds

def bo_tu_dung(cau):
    return [None if len(w) < DAI_MIN or w in TU_DUNG or la_so_la_ma(w) else w for w in cau]

def ghep_cum_tu(day, cum):
    kq, i = [], 0
    while i < len(day):
        a = day[i]
        b = day[i + 1] if i + 1 < len(day) else None
        if a is None:
            i += 1
        elif b is not None and (a, b) in cum:
            kq.append(a + "_" + b)
            i += 2
        else:
            kq.append(a)
            i += 1
    return kq

def tien_xu_ly(vb, cum, ds=None):
    ds = ds if ds is not None else chuan_bi(vb)
    kq = []
    for cau in ds:
        kq.extend(ghep_cum_tu(bo_tu_dung(cau), cum))
    return kq
```

Hình 2.1: Các hàm tiền xử lý



Hình 2.2: File B_48001 sau tiền xử lý



Hình 2.3: File R_43300 sau tiền xử lý

2.3. Cụm từ cố định

Một số cụm từ chỉ có nghĩa khi đi thành cặp: *nervous system* (hệ thần kinh), *blood pressure* (huyết áp), *new york*. Cách đơn giản nhất là lấy mọi cặp hai từ liền nhau (bigram) làm từ khóa, nhưng trên bộ dữ liệu này cách đó sinh ra thêm khoảng 865 nghìn từ khóa. Phần lớn là cặp ghép tình cờ, làm từ điển to lên đáng kể mà không thêm nhiều thông tin. Vì vậy chỉ giữ các cặp thật sự thường đi cùng nhau, đo bằng chỉ

số NPMI (Normalized Pointwise Mutual Information, thông tin tương hỗ theo điểm đã chuẩn hóa).

Ý tưởng: Gọi T là tổng số từ của bộ dữ liệu, $n(a)$ và $n(b)$ là số lần xuất hiện của từ a và từ b, $n(ab)$ là số lần a đứng ngay trước b. Xác suất tương ứng là

$$P(a) = \frac{n(a)}{T}, \qquad P(b) = \frac{n(b)}{T}, \qquad P(ab) = \frac{n(ab)}{T}.$$

Nếu a và b không liên quan đến nhau, về lý thuyết chúng chỉ đứng cạnh nhau khoảng mức tình cờ $E = \frac{n(a)n(b)}{T}$. Thực tế gấp mức tình cờ càng nhiều lần thì hai từ càng hay đi cùng nhau. Logarit của tỉ số này là PMI (Pointwise Mutual Information, thông tin tương hỗ theo điểm):

$$\mathrm{PMI}(a,b) = \ln \frac{P(ab)}{P(a)\,P(b)} = \ln \frac{n(ab)}{E}$$

PMI không có giới hạn trên nên khó đặt ngưỡng: Hai từ mà mỗi từ chỉ xuất hiện một lần và lần đó đứng cạnh nhau sẽ có PMI rất cao. Chia PMI cho $-\ln P(ab)$ để đưa nó về khoảng $[-1,1]$, ta được NPMI:

$$\mathrm{NPMI}(a,b) = \frac{\mathrm{PMI}(a,b)}{-\ln P(ab)}$$

NPMI bằng 1 khi hai từ luôn đi cùng nhau, bằng 0 khi chúng độc lập, bằng -1 khi chúng không bao giờ đứng cạnh nhau.

Ví dụ: Bảng 2.2 tính NPMI cho hai cặp từ với số liệu giả định, $T = 7{,}7$ triệu từ.

	nervous system	**great work**
$n(a)$	1.000	20.000
$n(b)$	5.000	15.000
$n(ab)$	357	60
$E = \frac{n(a)\,n(b)}{T}$	0,65	38,96
$\frac{n(ab)}{E}$	549,8	1,54
$\text{PMI} = \ln \frac{n(ab)}{E}$	6,31	0,43
$-\ln P(ab) = -\ln \frac{n(ab)}{T}$	9,98	11,76
$\text{NPMI} = \frac{\text{PMI}}{-\ln P(ab)}$	**0,63**:	**0,04**:

Bảng 2.2: Ví dụ tính NPMI

great work xuất hiện 60 lần, nhưng vì hai từ đều rất phổ biến nên mức tình cờ đã là khoảng 39 lần. *nervous system* gấp khoảng 550 lần mức tình cờ, nên đây đúng là một cụm từ có nghĩa.

Ngưỡng giữ cặp từ:

- NPMI $\geq$ 0,5: Điểm giữa của 0 và 1;
- Đứng cạnh nhau ít nhất 20 lần, để chặn các cặp hiếm có NPMI cao do tình cờ;
- Có mặt ở ít nhất 2 văn bản, để cụm từ không phải cách nói riêng của một cuốn sách.

Khi đếm, từ dừng và dấu câu là ranh giới, nên *house of lords* không bị ghép thành *house lord*.

```python
def tim_cum_tu(ds_vb, lan_min, vb_min, npmi_min):
    """Tìm cặp từ có NPMI >= npmi_min, gặp >= lan_min lần, ở >= vb_min văn bản."""
    n1, n2, df = Counter(), Counter(), Counter()            # n(a), n(ab), số văn bản có ab
    for vb in ds_vb:
        cap = set()
        for cau in vb:
            for a, b in zip(cau, cau[1:] + [None]):
                if a is None:
                    continue
                n1[a] += 1
                if b is not None:
                    n2[(a, b)] += 1
                    cap.add((a, b))
        df.update(cap)
    T = sum(n1.values())                                    # tổng số từ
    kq = []
    for (a, b), n in n2.items():
        if n < lan_min or df[(a, b)] < vb_min:
            continue
        p = n / T                                           # P(ab)
        npmi = math.log(p / (n1[a] / T * n1[b] / T)) / -math.log(p)   # PMI / -ln P(ab)
        if npmi >= npmi_min:
            kq.append((a, b, n, df[(a, b)], round(npmi, 3)))
    return sorted(kq, key=lambda x: -x[2])
```

Hình 2.4: Hàm tìm cụm từ cố định bằng NPMI

Kết quả: Tìm được **751 cụm từ**, so với khoảng 865 nghìn nếu lấy mọi bigram.

new york	eighteenth century	jesus christ
nervous system	small pox	venereal disease
symphony orchestra	british museum	national gallery
holy ghost	guinea pig	per acre

Bảng 2.3: Một số cụm từ cố định tìm được

2.4. Xây dựng từ điển

Sau tiền xử lý, mỗi văn bản là một danh sách từ khóa, trong đó một từ có thể lặp lại nhiều lần. Từ điển V là danh sách tất cả các từ khóa khác nhau xuất hiện trong toàn bộ 1.080 văn bản, mỗi từ chỉ ghi một lần và được đánh số thứ tự:

$$V = \{t_1, t_2, \ldots, t_M\}$$

với M là số từ trong từ điển.

Ví dụ: Giả sử bộ dữ liệu chỉ có 3 văn bản đã tiền xử lý:

- d_1: *patient symptom fever patient*

- d_2: *opera composer opera*

- d_3: *patient diet crop*

Gom mọi từ lại và bỏ các từ lặp, ta được từ điển gồm $M = 7$ từ:

$$V = \{composer, crop, diet, fever, opera, patient, symptom\}.$$

Cắt tỉa: Từ điển bỏ những từ chỉ có mặt ở đúng một văn bản. Các từ này thường là tên riêng, thuật ngữ tự đặt hoặc lỗi in của riêng cuốn sách đó, và không giúp tìm thấy văn bản nào khác ngoài chính văn bản chứa nó.

Bước	**Số từ khác nhau**	**Tổng số từ**
Văn bản sạch, tách theo khoảng trắng	564.309	16.039.319
Chữ thường, chỉ giữ chữ cái	148.829	16.080.420
Đưa về dạng gốc	134.434	16.080.420
Bỏ từ dừng, từ 1 – 2 chữ cái, số La Mã	133.633	7.715.574
Ghép 751 cụm từ cố định	134.359	7.662.066
Bỏ từ chỉ có ở 1 văn bản	60.129	7.539.891

Bảng 2.4: Kích thước từ điển sau từng bước

Từ bảng trên, ta thấy bỏ từ dừng giảm hơn một nửa tổng số từ (từ 16 triệu xuống 7,7 triệu) nhưng gần như không giảm số từ khác nhau: Từ dừng chỉ có 224 từ nhưng lặp lại rất nhiều lần. Ngược lại, cắt tỉa giảm số từ khác nhau hơn một nửa nhưng chỉ bỏ đi khoảng 1,6% tổng số từ, vì mỗi từ bị bỏ chỉ có ở một văn bản.

2.5. Trọng số TF-IDF

Cho bộ dữ liệu gồm N văn bản $d_1, d_2, \ldots, d_N$ và từ điển $V = \{t_1, \ldots, t_M\}$. Với mỗi từ t_i và mỗi văn bản d_j, cần một trọng số w_{ji} cho biết t_i quan trọng đến mức nào đối với nội dung của d_j. Trọng số này được tính từ hai thông tin sau:

- $\mathrm{TF}(t_i, d_j)$: Số lần t_i xuất hiện trong d_j.

- $\mathrm{DF}(t_i)$: Số văn bản có chứa t_i, tức số phần tử của tập $\{d_j : t_i \in d_j\}$. Từ có mặt ở càng nhiều văn bản thì càng ít giúp phân biệt văn bản này với văn bản khác.

Trọng số TF-IDF lớn khi TF lớn và DF nhỏ: Từ xuất hiện nhiều trong văn bản đang xét nhưng ít có mặt ở các văn bản khác.

1. Tần suất nghịch đảo văn bản (IDF)

IDF đo mức độ hiếm của một từ trong bộ dữ liệu:

$$\mathrm{IDF}(t_i) = \ln \frac{N}{\mathrm{DF}(t_i)}$$

Từ càng hiếm thì IDF càng lớn; từ có mặt ở mọi văn bản có IDF bằng 0.

2. Trọng số TF-IDF

Trọng số của từ t_i trong văn bản d_j là tích của TF và IDF:

$$w_{ji} = \mathrm{TF}(t_i, d_j) \times \mathrm{IDF}(t_i)$$

Ví dụ: Bộ dữ liệu có $N = 1.000$ văn bản. Trong văn bản d_1, ba từ *rabbit*, *said* và *time* đều xuất hiện 8 lần. *rabbit* có mặt ở 20 văn bản, *said* ở 900, *time* ở cả 1.000 văn bản.

Từ	**TF**	**DF**	**IDF** $= \ln(N/\mathbf{DF})$	$w = \mathbf{TF} \times \mathbf{IDF}$
rabbit	8	20	$\ln 50 \approx 3{,}91$	31,30
said	8	900	$\ln 1{,}11 \approx 0{,}11$	0,84
time	8	1.000	$\ln 1 = 0$	0

Bảng 2.5: Ví dụ tính trọng số TF-IDF

Cùng xuất hiện 8 lần, nhưng *rabbit* có trọng số gấp gần 40 lần *said*, còn *time* không có trọng số.

3. TF dạng logarit (sublinear TF)

Với TF thường, trọng số tăng tỉ lệ thuận với số lần xuất hiện. Sau tiền xử lý, mỗi văn bản còn khoảng 7.000 từ khóa nhưng chỉ khoảng 2.100 từ khác nhau, nên một số từ lặp lại hàng chục, thậm chí hàng trăm lần. Những từ này có thể chiếm gần hết độ dài vector, còn các từ khác dù cũng nói về nội dung lại có trọng số rất nhỏ.

Trong khi đó, mức quan trọng của một từ không tăng tỉ lệ với số lần lặp: Từ xuất hiện 2 lần thay vì 1 lần cho biết thêm nhiều về nội dung văn bản, còn 101 lần thay vì 100 lần thì gần như không thêm gì.

Cách khắc phục: Lấy logarit của TF. Logarit tăng chậm dần: TF bằng 1, 10, 100 cho $1 + \ln \text{TF}$ lần lượt bằng 1; 3,3; 5,6. Từ xuất hiện nhiều vẫn có trọng số lớn hơn, nhưng không lấn át các từ khác:

$$\text{TF}'(t_i, d_j) = \begin{cases} 1 + \ln \text{TF}(t_i, d_j) & \text{nếu TF}(t_i, d_j) > 0 \\ 0 & \text{nếu TF}(t_i, d_j) = 0 \end{cases}$$

Ví dụ: Trong một văn bản tôn giáo, *god* xuất hiện 100 lần và *prophet* xuất hiện 5 lần. Với TF thường, trọng số của *god* gấp 20 lần *prophet*; với TF dạng logarit, chỉ còn gấp khoảng 2,1 lần (5,61/2,61).

Trọng số hệ thống sử dụng là:

$$w_{ji} = \text{TF}'(t_i, d_j) \times \text{IDF}(t_i)$$

4. Chuẩn hóa độ dài

Một văn bản dài 7.000 từ có nhiều từ khác nhau hơn và mỗi từ lặp lại nhiều lần hơn một đoạn truy vấn dài 1.000 từ, nên vector của nó dài hơn nhiều. Nếu so sánh trực tiếp, văn bản dài luôn được điểm cao hơn dù nội dung không giống truy vấn hơn. Điều cần so sánh là tỉ lệ giữa các từ, tức nội dung, không phải độ lớn. Vì vậy mỗi vector được chia cho độ dài của nó (chuẩn hóa L_2):

$$x_{ji} = \frac{w_{ji}}{\sqrt{\sum_{k=1}^{M} w_{jk}^2}}$$

Sau bước này mọi vector có độ dài bằng 1, còn hướng của vector, tức tỉ lệ giữa các từ được giữ nguyên.

Kết quả:

Chủ đề	10 từ khóa có trọng số trung bình cao nhất
Triết học – Tâm lý – Tôn giáo	jesus, christ, gospel, sin, scripture, jesus christ, holy, spiritual, god, apostle
Khoa học xã hội	wage, industrial, economic, employer, social, trade, railroad, industry, worker, population
Mỹ thuật	painting, painter, drawing, portrait, paint, artist, decoration, panel, landscape, decorative
Y học	patient, symptom, medical, physician, disease, stomach, medicine, chronic, inflammation, hospital
Nông nghiệp	crop, manure, grower, plant, seedling, soil, planting, orchard, moisture, nut
Âm nhạc	composer, opera, musical, orchestra, piano, beethoven, violin, singer, music, melody

Bảng 2.6: Từ khóa đặc trưng của từng chủ đề

2.6. Biểu diễn văn bản trong không gian vector

Gọi $D = \{d_1, d_2, \ldots, d_N\}$ là bộ dữ liệu gồm $N = 1.080$ văn bản, và $V = \{t_1, t_2, \ldots, t_M\}$ là từ điển gồm $M = 60.129$ từ khóa. Mỗi văn bản d_j được biểu diễn bằng một vector M chiều:

$$\mathbf{x}_j = (x_{j1}, x_{j2}, \ldots, x_{jM}) \in \mathbb{R}^M,$$

trong đó x_{ji} là trọng số TF-IDF (đã chuẩn hóa) của từ t_i trong văn bản d_j. Nếu văn bản d_j không chứa từ t_i thì $x_{ji} = 0$.

Xếp vector của các văn bản thành từng hàng, ta được ma trận đặc trưng:

$$X = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_N \end{bmatrix} \in \mathbb{R}^{N \times M}.$$

Mỗi hàng là một văn bản, mỗi cột là một từ khóa của từ điển.

Ma trận thưa: Trong thực tế, mỗi văn bản chỉ chứa khoảng 2.100 trong số 60.129 từ

của từ điển, nên khoảng 96,5% phần tử của X bằng 0. Lưu toàn bộ ma trận sẽ tốn chỗ cho các số 0 và làm phép tính chậm.

Lưu trữ bằng chỉ mục đảo: Vì vậy ma trận X được lưu dưới dạng chỉ mục đảo. Chỉ mục đảo không thay đổi mô hình vector, mà chỉ là cách lưu ma trận X theo cột và bỏ các số 0: Mỗi cột của X ứng với một từ khóa t_i, và được lưu thành danh sách các văn bản có $x_{ji} \neq 0$ kèm giá trị x_{ji}. Chỉ mục đảo gồm hai phần:

- **Từ điển:** Mọi từ khóa của V, mỗi từ kèm IDF của nó, dùng để tính vector của văn bản truy vấn.
- **Danh sách văn bản:** Với mỗi từ khóa t_i, các cặp $(d_j,\ x_{ji})$ có $x_{ji} \neq 0$.

Các phần tử không được ghi đều bằng 0, và vector $\mathbf{x}_j$ của văn bản d_j có thể dựng lại bằng cách gom các cặp có d_j trong mọi danh sách.

2.7. Độ tương đồng cosine

Độ tương đồng cosine đo mức cùng hướng của hai vector trong không gian. Khi hai văn bản có nội dung giống nhau, chúng dùng cùng các từ khóa với tỉ lệ gần giống nhau, nên vector đặc trưng của chúng chỉ về gần cùng một hướng: Góc giữa hai vector nhỏ và cosine của góc tiến gần tới 1.

Độ tương đồng cosine giữa vector truy vấn $\mathbf{q}$ và vector văn bản $\mathbf{x}_j$ trong bộ dữ liệu được tính bằng:

$$\text{sim}(\mathbf{q}, \mathbf{x}_j) = \frac{\mathbf{q} \cdot \mathbf{x}_j}{\|\mathbf{q}\| \, \|\mathbf{x}_j\|} = \frac{\sum_{i=1}^{M} q_i x_{ji}}{\sqrt{\sum_{i=1}^{M} q_i^2} \, \sqrt{\sum_{i=1}^{M} x_{ji}^2}}$$

Các bước tính cụ thể:

- **Tích vô hướng:** Tổng tích các thành phần tương ứng của hai vector. Mỗi từ khóa chung đóng góp $q_i x_{ji}$, càng lớn khi từ đó càng quan trọng ở cả hai văn bản. Giá trị này thể hiện mức trùng nhau về từ khóa giữa hai văn bản.
- **Độ dài vector:** Tính độ dài của từng vector rồi chia tích vô hướng cho tích hai độ dài. Phép chia này giúp kết quả chỉ phụ thuộc vào nội dung (tỉ lệ giữa các từ khóa), không phụ thuộc văn bản dài hay ngắn.
- **Kết quả:** Giá trị nằm trong khoảng $[0, 1]$, vì mọi trọng số TF-IDF đều không âm. Giá trị càng gần 1 thì nội dung hai văn bản càng giống nhau, bằng 0 khi hai văn bản không có từ khóa chung nào.

Do các vector đã được chuẩn hóa độ dài bằng 1 từ trước, mẫu số luôn bằng 1, nên khi tính toán chỉ cần tích vô hướng và chỉ trên các từ có mặt ở cả hai văn bản:

$$\text{sim}(\mathbf{q}, \mathbf{x}_j) = \sum_{t_i \text{ có trong cả } q \text{ và } d_j} q_i x_{ji}$$

2.8. Quy trình tìm kiếm văn bản

Quá trình tìm kiếm với một văn bản truy vấn mới gồm 4 bước:

1. **Tiền xử lý văn bản truy vấn:** Văn bản truy vấn đi qua đúng các bước tiền xử lý đã áp dụng cho bộ dữ liệu: Tách câu, tách từ, đưa về dạng gốc, chuyển về chữ thường, bỏ từ dừng và ghép cụm từ cố định theo cùng danh sách 751 cụm.
   Kết quả: Danh sách từ khóa của truy vấn.

2. **Vector hóa truy vấn:** Đếm số lần xuất hiện của từng từ khóa, tính trọng số TF-IDF với IDF lấy từ từ điển của bộ dữ liệu, rồi chuẩn hóa độ dài L_2. Từ khóa không có trong từ điển bị bỏ qua.
   Kết quả: Vector truy vấn $\mathbf{q}$.

3. **Tính độ tương đồng:** Độ tương đồng cosine chỉ phụ thuộc vào các từ khóa chung, nên thay vì duyệt toàn bộ ma trận, hệ thống dùng chỉ mục đảo:

   - Gán điểm ban đầu của mọi văn bản bằng 0;
   - Với mỗi từ khóa t_i của truy vấn, lấy danh sách các văn bản chứa t_i;
   - Với mỗi văn bản d_j trong danh sách, cộng $q_i x_{ji}$ vào điểm của d_j.

   Kết quả: Điểm của mỗi văn bản chính là độ tương đồng cosine của nó với truy vấn.

4. **Hiển thị kết quả:** Sắp xếp các văn bản theo độ tương đồng giảm dần.
   Kết quả: Lấy 3 văn bản có điểm cao nhất.